

CSCE 614 COMPUTER ARCHITECTURE · MODULE 06 · LECTURE NOTES

Cache Optimization and Prefetching

Restructuring code and data so the hierarchy works for you.

MODULE OUTCOMES

Apply loop interchange, blocking, and fusion for locality.

Choose between array-of-structs and struct-of-arrays.

Explain hardware prefetching and what defeats it.

Use software prefetch intrinsics and know when not to.

Predict a transformation's benefit before measuring it.

1 Loop transformations

1.1 Interchange

C stores arrays in row-major order, so $a[i][j]$ and $a[i][j+1]$ are adjacent in memory while $a[i][j]$ and $a[i+1][j]$ are N elements apart.

```
// Column-major traversal: stride N*4 bytes
for (j = 0; j < N; j++)
    for (i = 0; i < N; i++)
        sum += a[i][j];

// Row-major traversal: stride 4 bytes
for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
        sum += a[i][j];
```

Both perform exactly the same N^2 additions and produce the same result. The first fetches a 64-byte line and uses 4 bytes of it before moving on; the second uses all 64. That is sixteen times the memory traffic for identical work, and the measured difference is typically 5–10× (less than 16 because bandwidth, not latency, becomes the limit).

PREDICT BEFORE YOU MEASURE

With a 64-byte line and 4-byte floats, the bad version uses 1/16 of each line, so you should expect roughly an order of magnitude.

Making the prediction before running the benchmark is the habit this course is trying to build. When the prediction is right, your model of the machine is working; when it is wrong, you have learned something specific.

1.2 Blocking

Naive matrix multiplication traverses B column-wise, once for every row of A . For large N , B has been entirely evicted between passes, so nothing is reused.

Blocking (or tiling) restructures the computation to operate on sub-blocks small enough to remain resident while they are being reused.

```
for (ii = 0; ii < N; ii += B)
  for (jj = 0; jj < N; jj += B)
    for (kk = 0; kk < N; kk += B)
      for (i = ii; i < ii+B; i++)
        for (j = jj; j < jj+B; j++) {
          float s = c[i][j];
          for (k = kk; k < kk+B; k++)
            s += a[i][k] * b[k][j];
          c[i][j] = s;
        }
```

Choosing B . Three $B \times B$ tiles must fit in L1 simultaneously, so $3B^2 \times 4 \text{ bytes} < 32 \text{ KB}$, giving $B \approx 52$. In practice $B = 32$ or 48 works well, leaving room for other data. Each tile element is now loaded once and used B times, cutting misses by roughly a factor of B . On large matrices this is commonly a 3–5× speedup with no change to the arithmetic.

Transformation	What it does	Use when
Interchange	Reorders nested loops.	The innermost loop has the wrong stride.
Blocking / tiling	Operates on sub-blocks.	The working set exceeds a cache level and there is reuse to capture.
Fusion	Merges two loops over the same data.	Data would otherwise be read, evicted, and read again.
Fission	Splits one loop into several.	The body touches too many independent streams for the prefetcher or the cache to track.
Unrolling	Fewer iterations with a larger body.	Loop overhead is significant, or you need to break a dependency chain (Module 03).

Fusion and fission pull in opposite directions, which is a good reminder that these are not rules but responses to a measured diagnosis.

2 Data layout

Layout changes are frequently larger wins than loop transformations, and should be considered first.

2.1 Array of structs versus struct of arrays

```
// Array of structs -- natural object-oriented layout
struct Particle { float x,y,z, vx,vy,vz; int id; float mass; }; // 32B
Particle p[N];

for (i...) p[i].x += p[i].vx * dt;    // touches 8 of every 32 bytes

// Struct of arrays -- what the hardware wants for bulk work
struct Particles { float x[N], y[N], z[N], vx[N], vy[N], vz[N]; };

for (i...) P.x[i] += P.vx[i] * dt;    // two dense sequential streams
```

	Array of structs	Struct of arrays
Best when	You touch most fields of one object at a time.	You touch one or two fields across many objects.
Cache efficiency	Poor for bulk processing — fetches fields you will not use.	Excellent — every byte fetched is used.
Vectorisation	Difficult: values are strided.	Natural: consecutive values are contiguous (Module 09).
Ergonomics	Natural and readable.	Awkward; often wrapped in an accessor layer.

THIS IS WHY ENTITY-COMPONENT SYSTEMS EXIST

Game engines process thousands of objects by applying one operation to one field across all of them — integrate positions, cull bounds, update transforms. That is exactly the SoA access pattern.

ECS architectures are, at the data level, an organised way of getting struct-of-arrays layout while retaining a usable programming model. The performance argument is this cache argument, and nothing more mysterious.

The same reasoning governs vertex buffer layout in CSCE 641: interleaved attributes when the shader uses all of them, separate streams when different passes use different subsets.

2.2 Other layout techniques

- **Hot/cold splitting.** Move rarely accessed fields into a separate structure referenced by pointer. More hot objects fit per cache line.
- **Alignment.** Align frequently accessed structures to 64 bytes so a single object does not straddle two cache lines and cost two fetches.

- **Shrink the data.** float rather than double, 16-bit indices, quantised normals and colours. When memory-bound, halving the bytes often nearly halves the time — and this is usually easier than any loop transformation.
- **Replace pointers with indices.** A flat array indexed by 32-bit integers is smaller than one of 64-bit pointers and, more importantly, avoids the serialised dependent loads of Module 02.

3 Prefetching

Memory latency can be hidden entirely if the data is requested before it is needed. Processors attempt this automatically.

3.1 Hardware prefetchers

Pattern	Prefetched?	Why
Sequential forward or backward	Yes	Trivially detected; the common case.
Constant stride	Yes	Stride detectors handle regular strides well.
Multiple interleaved streams	Usually	Modern prefetchers track several streams at once, but there is a limit.
Random access	No	Nothing to detect.
Pointer chasing	No	The next address is unknown until the current load returns.
Indirect, <code>a[b[i]]</code>	No	The address depends on loaded data.

WHAT SUCCESSFUL PREFETCHING ACTUALLY ACHIEVES

It converts a **latency** problem into a **bandwidth** problem.

Instead of waiting 250 cycles for each line in turn, many requests are in flight simultaneously and the limit becomes how fast the memory system can deliver bytes — which is comparatively generous.

This is why sequential access is vastly faster than the latency table alone would suggest, and why the random pointer chase of Module 05 had to be randomised to measure latency at all.

Prefetchers stop at page boundaries, since prefetching into an unmapped page risks a spurious fault. This is one of the arguments for huge pages in Module 07.

3.2 Software prefetch

```
for (i = 0; i < N; i++) {
    __builtin_prefetch(&data[index[i + DISTANCE]], 0, 1);
    process(data[index[i]]);    // indirect -- hardware cannot predict
}
```

Explicit prefetch instructions are worth trying for exactly the patterns hardware cannot handle: indirect indexing, graph and tree traversal, hash table probing. The prefetch *distance* must be tuned — too short and the line has not arrived; too long and it is evicted before use — and the right value depends on the machine and on how much work each iteration does.

For sequential access, software prefetch almost always makes things slightly worse: the hardware was already handling it, and you have added instructions and cache pressure. Measure, always.

4 A method

Step	Question	How to answer it
1	Is this actually memory-bound?	IPC below ~1 with a high cache-miss rate. If IPC is 3, memory is not your problem and none of this will help.
2	Which level is missing?	L1, L2, and LLC miss counters separately. An L1 miss served by L2 costs 12 cycles; one served by DRAM costs 250.
3	Which of the three Cs?	Vary the working-set size (capacity) and the stride (conflict). Compulsory misses remain when neither changes anything.
4	Fix the layout.	AoS→SoA, hot/cold splitting, smaller types, indices instead of pointers.
5	Then fix the loops.	Interchange, block, fuse or split.
6	Then consider prefetching.	Only for patterns hardware cannot detect, and only with measurement.

LAYOUT BEFORE LOOPS

The ordering in that table is deliberate and frequently got backwards. People reach for tiling first because it feels like the sophisticated answer.

A layout change often removes the need for any loop transformation at all, is usually simpler, and compounds with everything else. Try it first.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Ulrich Drepper — What Every Programmer Should Know About Memory	Section 6 is a catalogue of exactly these optimisations with measured results. The core reading for this module.
Onur Mutlu — Prefetching lectures	Hardware prefetcher designs and their limitations.

Resource	What it covers for this module
Igor Ostrovsky — Gallery of Processor Cache Effects	Nine experiments isolating individual effects. Run every one.
Mike Acton — Data-Oriented Design and C++ (CppCon, free video)	The practitioner's argument for layout-first thinking, from a game engine perspective. Directly relevant to CSCE 641.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Measure row-major against column-major traversal of a large 2D array. Predict the ratio from the cache line size before measuring, then explain any discrepancy.
- 2 Implement naive and blocked matrix multiplication. Compute the optimal tile size from your L1 capacity, then sweep B and confirm the optimum empirically.
- 3 Convert a particle system from AoS to SoA. Measure an update that touches two fields of eight, and report both the speedup and the change in cache misses.
- 4 Apply hot/cold splitting to a struct where one field is accessed in a hot loop and six are not. Measure.
- 5 Compare a sequential array sum against a random-order sum of the same data. Then add software prefetch to the random version and report whether it helped.
- 6 Take a loop with an indirect access pattern (`a[b[i]]`) and tune a software prefetch distance. Plot time against distance and find the optimum.
- 7 Convert a double computation to float where precision permits. Measure, and determine whether you became compute-bound.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is column-major traversal of a row-major array 5–10× slower, and why not exactly 16×?
- 2 How do you compute the right tile size for blocked matrix multiplication?
- 3 When is array-of-structs the right choice, and when is struct-of-arrays?
- 4 Name three access patterns a hardware prefetcher cannot handle.
- 5 What does successful prefetching convert a latency problem into, and why does that help?
- 6 Why should layout changes be attempted before loop transformations?
- 7 What is the first thing to check before applying any optimisation in this module?